SOP University of Kentucky LCC GPU Resources

Purpose. This SOP gives Sapkota Lab members a repeatable, responsible workflow for requesting access to the University of Kentucky Lipscomb Compute Cluster LCC, selecting GPU resources, running analyses through Slurm, and preserving reproducibility.

Document field	Value
Owner	Sapkota Lab
Applies to	Lab members using UK CCS LCC resources
Version	1.0 | 7 September 2026
Authoritative source	Current CCS documentation and support portal

Read this first

LCC is a shared, scheduled system. Do not run computationally intensive work on a login node. Request GPUs only through an interactive or batch Slurm allocation; release them as soon as the task is complete. CCS currently describes LCC as a general-purpose Intel-based cluster with NVIDIA GPUs for accelerated machine-learning and scientific workloads. Hardware, partitions, limits, software modules, and account policies can change, so this SOP uses live discovery commands instead of fixed values.

Quick path

1 Access	2 Prepare	3 Request GPU	4 Record results
Account plus PI sponsor	Data plus environment	Interactive test then sbatch	Logs plus job ID plus versions

1 Access and onboarding

UK faculty, staff, and students can request CCS access; students must be sponsored by a faculty or research staff member. Before requesting an account, identify the sponsoring PI, project purpose, data sensitivity, anticipated CPU, memory, storage, and GPU needs, and software or container requirements.

1. Submit a CCS account request through the UK CCS support portal. Request LCC access and state that the project needs NVIDIA GPU resources for analysis.
2. Include the PI or supervisor, your LinkBlue ID, a concise project description, expected analysis software, approximate dataset size, expected GPU count and wall time, and whether you will use Open OnDemand or SSH.
3. Wait for account confirmation and instructions on group, storage, Duo, VPN, or acceptable-use requirements. Do not share accounts or credentials.
4. Start in the LCC Open OnDemand portal https://ood.ccs.uky.edu/ unless the project already has an established SSH workflow. CCS recommends Open OnDemand for new users.
5. If using SSH, follow the current CCS SSH Login Instructions rather than copying an old hostname from a lab note. Use a login node only to edit, transfer modest files, inspect jobs, and submit allocations.

2 Data and project setup

Use a dedicated project directory and keep input data, code, environments, logs, and results distinct. Use current CCS storage guidance to select home, project, and scratch storage. Treat scratch as temporary working space, not the only copy of results.

```
mkdir -p ~/projects/<project>/{code,data,env,logs,results}
cd ~/projects/<project>
git init                # if permitted and appropriate
date; hostname; pwd
```

For large transfers, use CCS-supported tools such as Globus rather than repeatedly copying data through a login session. Do not put regulated, confidential, export-controlled, FERPA, HIPAA, or other sensitive data on LCC until the PI confirms that the data class and CCS environment are approved.

Reproducibility minimum. Keep a README with the input-data source/version, exact command, Git commit or script hash, module list or container digest, package environment export, Slurm script, job ID, and output location.

3 Discover the live GPU configuration

Never guess a partition name, GPU type, GPU count per node, or time limit. After logging in, inspect the live Slurm configuration and choose resources that match the workload.

```
sinfo -o "%P %a %l %D %G %C"
scontrol show partition
squeue -u "$USER"
sacctmgr show assoc user="$USER" format=User,Account,Partition,QOS
module avail 2>&1 | less
```

In sinfo output, %G exposes generic resources such as GPUs. Partition information reveals names and limits. Your account/QOS association determines what you can request. If GPUs do not appear, open a CCS support request with the command output and project need.

4 Request a GPU allocation

Interactive allocation for a short validation test

Replace bracketed values only with live values permitted for your account. Use a short interactive allocation to validate the environment; it is not a substitute for a batch job.

```
srun --pty --partition=<gpu_partition> --account=<account> \
  --gpus=1 --cpus-per-task=4 --mem=32G --time=00:30:00 bash
nvidia-smi
python --version
python -c "import torch; print(torch.cuda.is_available());
print(torch.cuda.get_device_name(0))"
```

If your site requires a different Slurm GPU syntax, use the syntax confirmed by current CCS documentation. Do not work around scheduling by launching compute on a login node.

Batch job template

Save as submit_gpu.sh. Start small and use logs and Slurm accounting to right-size later jobs.

```
#!/bin/bash
#SBATCH --job-name=<analysis_name>
#SBATCH --partition=<gpu_partition>
#SBATCH --account=<account>
#SBATCH --gpus=1
```

```
#SBATCH --cpus-per-task=4
#SBATCH --mem=32G
#SBATCH --time=02:00:00
#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err

set -euo pipefail
mkdir -p logs results
module purge
module load <module_or_conda>
echo "Started: $(date)"; hostname; nvidia-smi
python code/run_analysis.py --input data/<input> --output results/<run_name>
echo "Finished: $(date)"
```

```
chmod 700 submit_gpu.sh
sbatch submit_gpu.sh
squeue -u "$USER"
scontrol show job <job_id>
```

Open OnDemand option

For notebooks, graphical analysis, or a first interactive workflow, log into LCC Open OnDemand. Launch the available Jupyter, RStudio, desktop, or shell app; select only needed GPU, memory, CPUs, and wall time; submit; then connect once the job starts. Work inside the allocated session and end it when finished.

5 Monitor validate and finish

- Before a production run, confirm GPU visibility with nvidia-smi and run a small known input. Confirm output is written to the intended project location.
- Monitor work with squeue -u "$USER". Read .out and .err logs; completion does not equal scientific correctness.
- After completion, inspect use with sacct -j <job_id> --format=JobID,JobName,State,Elapsed,AllocCPUS,ReqMem,MaxRSS,AllocTRES. Field availability can vary.
- Cancel accidental or no-longer-needed jobs with scancel <job_id>. End Open OnDemand sessions when done.
- Copy final irreplaceable results to approved long-term project storage. Clean temporary scratch intermediates under current CCS retention guidance.

6 Resource stewardship and troubleshooting

Situation	Action
Job stays pending	Run squeue -j <job_id> and scontrol show job <job_id>. Review Reason; reduce the request when scientifically acceptable or ask CCS to clarify limits.
GPU not visible	Verify you are in a scheduled GPU allocation; run nvidia-smi; load documented CUDA/software modules. Include job ID and output in a support request.
Out of memory or time	Use sacct and application logs. Increase only the constrained resource; do not request a full node by default.
Package conflict	Start with module purge, then use a documented module, conda environment, or container. Record the exact environment.

Need a different GPU or more scale	Consult CCS early. Provide framework, dataset size, GPU-memory need, GPU count, runtime, and multi-GPU capability.

Lab run record

Complete one record per production analysis. Store it with the project README or electronic lab notebook.

Record item	Enter here
Researcher and date	
Project and PI	
Input dataset and version	
Code repository and commit	
Slurm job ID and script path	
Requested resources and partition	
Software modules environment or container	
Results path validation notes and cost of rerun	

7 Sapkota Lab LCC operating profile

This section is based on the lab's checked-in LCC job scripts and project code. It is the working default for the two projects below, not a substitute for live LCC policy. The lab account embedded in current production scripts is gol_msa520_uksr. Confirm that it remains your authorized allocation before submitting. Never replace an account name or partition simply because it appears in an older script.

Workload class	Project validated LCC pattern	Use it for
V100 32 GB GPU	V4V32_SKY32M192_L; 1 GPU; 8 CPUs; 32–64 GB RAM; 1–4 hours	Routine stomatal inference, mask proposals, and post-model imaging.
A100 80 GB GPU	A2V80_ICE56M256_L; 1 GPU; 8 CPUs; 64 GB RAM; 1–8 hours	Fine-tuning, calibration, held-out validation, and external domain experiments.
CPU batch or job array	Use a current LCC CPU partition selected with sinfo; do not request a GPU by default	SeedThermal FLIR radiometric extraction, masking, and summary statistics.
Interactive portal	Open OnDemand with minimal requested resources	Notebook inspection, small tests, log review, and one-image smoke tests.

The production scripts use ccs/conda/python-3.10.12 and ccs/cuda/12.2.0_535.54.03, followed by the project .venv. Confirm both module names with module avail on the day of use. Module availability and driver compatibility are cluster-controlled.

8 Stomatal Phenotyping Pipeline on LCC

Why this project uses GPUs. The StomatalPhenotypingPipeline trains and executes object-detection and segmentation models on bright-field stomatal images. GPU allocation is appropriate for YOLO/StomaDet model training, tiled full-resolution inference, and SAM-based complex-mask proposals. Dataset construction, CSV joins, report generation, and scientific review remain CPU tasks and should not occupy a GPU.

Expected project layout and data flow

Stage	Inputs	Primary output and gate
1 Prepare	Stomata_Images; box_annotations; image_metadata.csv; combined_study_manifest.csv	A complete project copy and an activated .venv. Keep raw images read-only.
2 Train or validate	training_data/annotated_v1; public external benchmark where applicable	training_runs/<run>/weights/ best.pt plus held-out validation artifacts.
3 Calibrate	Low-threshold predictions and held-out validation labels	Calibration report. Select thresholds on validation only; do not tune on the test split.
4 Production inference	Model, calibration report, all in-scope images, combined manifest	raw_predictions, calibrated_detections.csv, stomata.csv, and run_manifest.json.
5 Biological interpretation	QC-approved thermal and stomatal records	Integrated tables and reports; models do not replace image-level quality control.

One-time environment setup

```
cd /path/to/StomatalPhenotypingPipeline
module --force purge
module load ccs/conda/python-3.10.12
module load ccs/cuda/12.2.0_535.54.03
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install --no-build-isolation -r requirements.txt
python download_model.py
```

Run setup from an LCC login node only. Do not run inference there. Some model checkpoints are first downloaded from Hugging Face; if compute nodes have no outbound network access, stage the checkpoint/cache before submitting a GPU job, following CCS policy.

Approved job choices

Need	Use this checked-in script	Expected resource choice
Production tomato detection	cluster/ lcc_production_inference.sbatch	V100 32 GB, one GPU, 8 CPUs, 64 GB RAM, 2 hours.
Tomato detector fine-tuning	cluster/lcc_finetune.sbatch	A100 80 GB, one GPU, 8 CPUs, 64 GB RAM, 8 hours.
Held-out calibration	cluster/	A100 80 GB, one GPU, 8 CPUs, 64

	lcc_full_image_calibration.sbatch	GB RAM, 2 hours.
Complex-mask proposals	cluster/ lcc_complex_mask_proposals.sbatch or lcc_complex_masks_all.sbatch	V100 32 GB, one GPU, 8 CPUs, 32 GB RAM, 1 hour.
Independent test evaluation	cluster/ lcc_validate_annotated.sbatch	A100 80 GB, one GPU, 8 CPUs, 32 GB RAM, 1 hour.
External-domain research	cluster/ lcc_external_domain_adaptation.sbatch or lcc_stomadet_external.sbatch	A100 80 GB; outputs remain research artifacts unless the stated held-out gate passes.

Do not use an external-domain or pseudo-label experiment to replace production tomato counts merely because it ran successfully. The project documentation explicitly requires a passing untouched accuracy gate; the stability audit is secondary and cannot establish accuracy by itself.

Production execution checklist

- Confirm prerequisite files with test -s before submit: model weights, split manifest, calibration report, and combined_study_manifest.csv.
- Read every #SBATCH directive. Confirm the account, partition, time, memory, output paths, and one-GPU request against live sinfo and your allocation.
- Submit the checked-in script rather than typing a long one-line command: sbatch cluster/lcc_production_inference.sbatch.
- After completion, inspect validation_artifacts/tomato_production_v1/run_manifest.json. It records the SLURM job ID, Git commit/dirty status, model hash, calibration hash, input-manifest hash, tile size, overlap, and ledger counts.
- Review outputs before downstream analysis: raw predictions, calibrated detections, the prediction ledger, and image-level QC. Archive the Slurm .out/.err files alongside the run manifest.

9 SeedThermal workflow on LCC

Why this project normally does not need a GPU. SeedThermal reads radiometric FLIR ONE Edge JPGs, extracts per-pixel Celsius arrays, applies fixed ROIs or manually approved binary plant masks, and writes plant-level temperature summaries. Its core packages are flyr, NumPy, and Pillow; its checked-in implementation does not use CUDA. LCC helps when there are many images or sessions to process, but request CPU resources unless a future version adds validated GPU acceleration.

SeedThermal input and output contract

Component	Lab convention	Required check
Raw imagery	Radiometric FLIR Ignite JPGs; expected names include SYYYYMMDD_P##_genotype.JPG	Keep source JPGs unchanged and retain capture/session identifiers.
Metadata	studies/tomato_2026/ pair_metadata.csv and session/observation records	Each image position must match its left/right treatment and variety metadata.
Masks	{image_stem}_{left|right}_mask.png	Mask selected-pixel count must meet the configured minimum;

		failed masks require review.
Plant-level outputs	processed/plant_analysis/<analysis-id>/ and thermal CSV tables	Check mean, median, p10, p90, pixel_count, and QC flags before combining studies.
Integrated outputs	combined study manifest and downstream stomatal joins	Join only stable plant-unit keys; do not infer matches from file order.

Local smoke test then LCC batch

```
# Verify the file inventory without processing
python scripts/run_thermal_phenotype.py   --input <radiometric_jpg_directory>   --plate-id
<session_label> --dry-run

# Run pair-aware extraction after masks and metadata pass review
python scripts/run_masked_pair_analysis.py   --input <radiometric_jpg_directory>   --pair-
metadata studies/tomato_2026/pair_metadata.csv   --mask-dir <approved_mask_directory>   --
analysis-id <session_mask_version>
```

Use one local image first to validate flyr, temperature orientation, file naming, metadata lookup, and mask alignment. Only then scale on LCC. A failed thermal run must preserve its manifest and error report; do not silently drop unreadable images or masks with too few selected pixels.

CPU array template for many independent FLIR images

The project does not currently include an LCC SeedThermal job script, so use this template only after substituting a live, authorized CPU partition and account. It demonstrates the correct CPU pattern: one image/session per task, no GPU request. For paired-mask analyses that need a shared output manifest, run one batch job after all approved masks are present instead of parallel writers targeting the same file.

```
#!/bin/bash
#SBATCH --job-name=seedthermal
#SBATCH --account=<authorized_lab_account>
#SBATCH --partition=<live_cpu_partition>
#SBATCH --cpus-per-task=2
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH --array=0-<N_MINUS_1>
#SBATCH --output=logs/seedthermal-%A_%a.out
#SBATCH --error=logs/seedthermal-%A_%a.err

set -euo pipefail
cd <SeedThermal_project_root>
source .venv/bin/activate
mapfile -t IMAGES < <(find <radiometric_jpg_directory> -maxdepth 1 -iname '*.jpg' | sort)
python scripts/run_thermal_phenotype.py   --input "$(dirname "$
{IMAGES[$SLURM_ARRAY_TASK_ID]}")"   --plate-id "$(basename "$
{IMAGES[$SLURM_ARRAY_TASK_ID]}" .JPG)"   --roi-config <approved_roi_json>   --output-root
processed/runs
```

Before using this array example, adapt the CLI to guarantee one task processes one unique input/output unit. The current run_thermal_phenotype.py takes a directory as input, so submitting several tasks against the same directory would duplicate work. A safer first LCC run is one CPU batch job per session directory, or a project-reviewed wrapper that supplies unique file lists.

Thermal quality-control gates

- Validate radiometric decoding on at least one representative image before a full session. A visually plausible JPG is not proof that the temperature grid decoded correctly.

- Review the naming convention, session date, position, and genotype parsed from each filename. Unparseable files must be flagged, not assigned by guesswork.
- For masked extraction, confirm the left/right mask matches the image stem, treatment assignment, and plant side; enforce the minimum selected-pixel check.
- Treat canopy temperature as a derived phenotype: retain source file, capture time, mask/ROI version, pixel count, all summary statistics, and QC flag in the final table.
- Before joining to stomatal measurements, use the combined-study manifest and verify treatment, variety, plant unit, and session identifiers. Do not join on a display label alone.

10 Submission and closeout protocol

6. Preflight: clean or document the Git worktree; create a run-specific output directory; check data availability and required model/mask files; run the smallest valid input first.
7. Submit: record script path, commit hash, account, partition, requested resources, job ID, and intended output location in the lab run record at submission time.
8. Observe: inspect squeue and the first log lines. Confirm the assigned GPU with nvidia-smi for GPU jobs. Stop incorrect jobs early with scancel <job_id>.
9. Validate: test scientific gates, not just exit code. For stomata, inspect held-out/calibration outputs and QC; for thermal, inspect decoded temperatures, masks, metadata, and QC flags.
10. Close out: retain logs, job script, environment/module list, manifest, and immutable inputs or checksums with results. Transfer final results from scratch to approved project storage.

References and support

UK CCS Research Computing: https://www.ccs.uky.edu/services-and-support/research-computing/ — LCC overview, GPUs, Open OnDemand, Slurm, data transfer, account and support links.

UK CCS New to CCS Getting Started: https://www.ccs.uky.edu/services-and-support/new-to-ccs-getting-started/ — account path, new-user documentation, and sponsorship.

UK CCS Training and Consulting: https://www.ccs.uky.edu/services-and-support/training-consulting/ — consultation, reproducibility, training, and TensorFlow on LCC guidance.

LCC Open OnDemand portal: https://ood.ccs.uky.edu/ — browser-based LCC access.

Source check: 7 September 2026. Consult linked CCS documentation and the support portal before using example values, because cluster details and policy can change.